MAEG4060 Virtual Reality Systems and Applications

Course Project — Final Stage Report

The Chinese University of Hong Kong (CUHK)

Project title:

Interactive Physical Simulation of Off-Road Vehicle Dynamics
in a Mountainous Terrain

Group members

Guo Tsun Hei (1155186272) — 3D modeling, texturing, rigging, animation

Yim Fu Chong (1155176974) — Programming, physics, system integration

Submission deadline: 17 April 2026

Abstract. This project is a DirectX 9.0c sandbox (not a scored game) where a player drives an off-road vehicle on procedural mountainous terrain. The build supports keyboard and XInput gamepad control, third-person chase camera, terrain splatting, collision with static and dynamic props, windmill animation, and a day/night lighting cycle. This report is intentionally concise: deliverables first, then implementation and authorship evidence. Wording, figure captions, and in-text file paths are written as a single technical record—none are ornamental.

Contents

1 Project snapshot

1.1 What is delivered

- Executable package: `Game_Build/maeg4060_stage2.exe` with `Game_Build/Assets/` (*generated locally; not tracked in git*).
- Source code: `src/` and `CMakeLists.txt`.
- Documentation: Stage PDFs (`docs/stageOne.pdf`, `docs/stageTwo.pdf`, `docs/stageFinal.pdf`) plus the **standalone user guide** `docs/User_Guide.pdf` (controls, requirements, and troubleshooting; summarized in §2).
- Public repository (for markers, version control, and CI): `https://github.com/Rex-Yim/Direct3DTerrainModels`.
- Media: figures in this PDF; short viewport recordings under `docs/media/`; README GIFs under `docs/images/`.

1.2 Course expectations mapping

Table 1 maps typical final-stage submission themes from the course project brief to sections of this report, companion PDFs, and code entry points.

Theme	Where it is covered
Input (keyboard / XInput)	§2; implementation in `src/Input.cpp`
Terrain height & vehicle following	§3.2; `src/Terrain.cpp`, `src/Vehicle.cpp`
Collision & physics-based motion	§3.3; `src/Collision.cpp`, `src/Vehicle.cpp`, boulder logic in `src/Game.cpp`
Animation / hierarchy models	§3.4; `src/AnimatedModel.cpp`, windmill update/draw in `src/Game.cpp`
Authorship & media evidence	§4; assets under `Assets/models/`
Build, D3DX, packaging	§5; `CMakeLists.txt`, Appendix-style notes in §5
User guide (separate PDF)	`docs/User_Guide.pdf` (full detail); §2 for a short extract

Table 1: *Index for marking: themes vs. sections and code.*

1.3 Core features

- Procedural heightfield terrain with bilinear height sampling.
- Grass base plus slope/height-driven splat layers (rock, stone, mountain).
- Vehicle driving with steering, throttle/brake, and terrain-normal alignment.
- Windmill: hierarchical `.x` with `ID3DXAnimationController` when keyframed data is present; static mesh path uses subset draws / code-driven motion when animation sets are absent (§3.4).
- Collision with crate (AABB) and dynamic boulder (sphere, gravity, impulse).
- HUD: smoothed FPS, speed, world position, simulation clock, windmill mode (animation vs. static spin vs. paused), terrain mode/splat summary, optional mini-map/legend; plus day/night lighting.

Figure 1: *In-application view: procedural splats, windmill, vehicle, props, and HUD.*

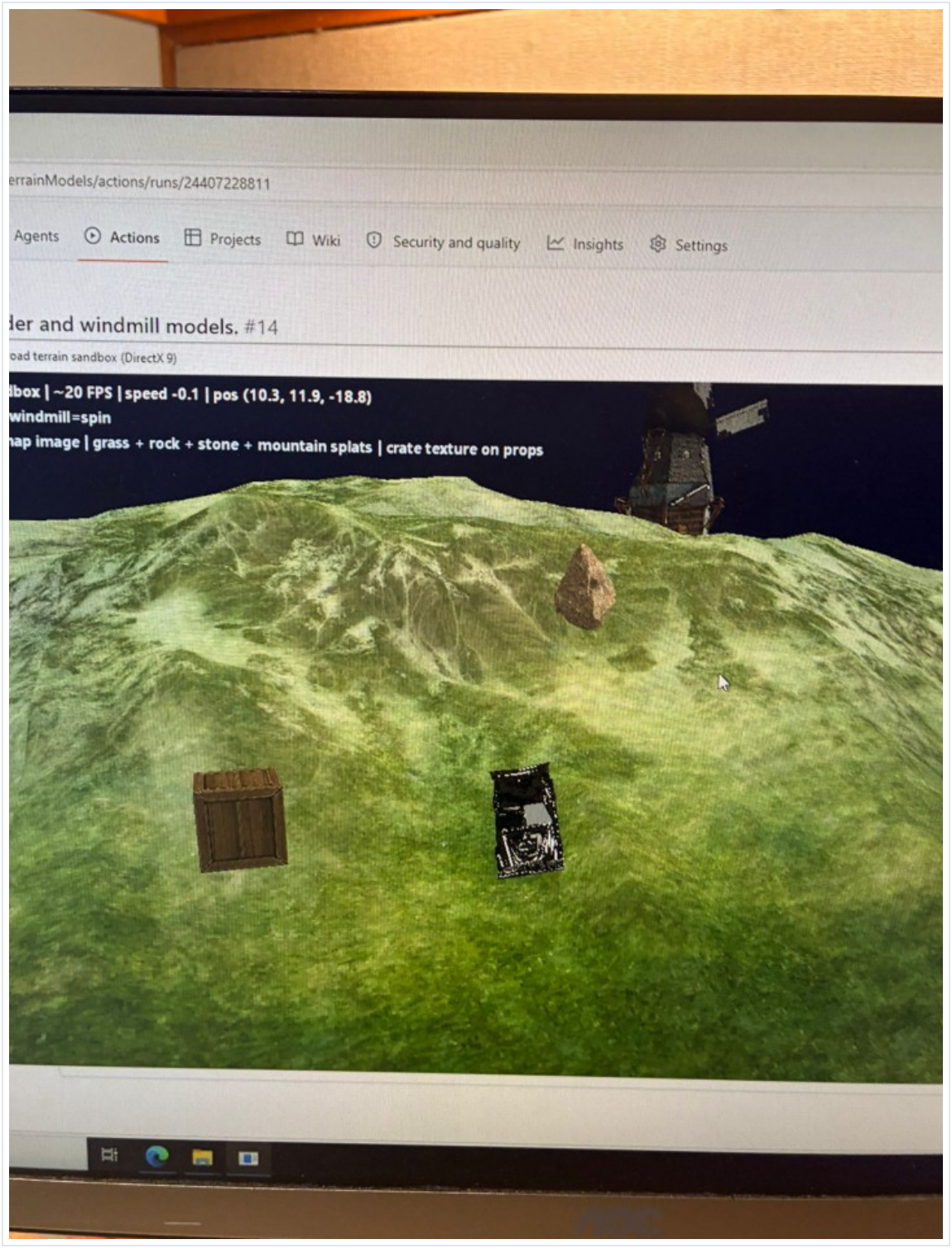


Figure 2: *GitHub Actions workflow run showing the integrated sandbox result.*

1.4 Runtime screenshots

2 How to run and evaluate

2.1 System requirements

Windows 10 or later (64-bit recommended), DirectX 9 compatible GPU, display at least 1280×720. Optional Xbox-compatible controller (XInput).

2.2 Launch

Run `maeg4060_stage2.exe` from `Game_Build` with `Assets` beside the executable. Build details are summarized in §5. The **standalone user guide** deliverable (`docs/User_Guide.pdf`) contains the authoritative control list, requirements, and troubleshooting; this section duplicates only the essentials.

2.3 Controls

Input	Action
W / S	Throttle forward / reverse
A / D	Steer left / right
Space	Brake
M	Pause / resume windmill animation
Esc	Exit application

Gamepad: left stick steers; right trigger throttles; left trigger brakes / partial reverse.

2.4 Quick troubleshooting

- Missing D3DX DLL: install the legacy DirectX end-user runtime, or ship `D3DX9_43.dll` next to the exe when the build copies it.
- Missing models or textures: keep `Assets/` next to the executable (working directory must resolve relative paths).
- Blank screen after window resize: the sample calls font and device lost/reset around `IDirect3DDevice9::Reset`—confirm GPU drivers; see user guide if issues persist.

3 Implementation summary

3.1 Frame pipeline

The loop is input → update → render. Input uses `GetAsyncKeyState` and `XInputGetState`; update integrates vehicle and boulder motion, resolves collisions, advances animation timing, and updates lighting; render draws terrain, models, and HUD via the fixed-function Direct3D 9 pipeline.

3.2 Terrain and navigation

Terrain height uses bilinear interpolation:

$$h(x,z) = (1-t_x)(1-t_z)h_{00} + t_x(1-t_z)h_{10} + (1-t_x)t_z h_{01} + t_x t_z h_{11}.$$

The vehicle aligns its up vector with the local terrain normal; steering sets yaw for stable hill following.

3.3 Physics and collision

Crate: AABB tests. Boulder: bounding sphere with gravity and simple impulses on vehicle contact (lightweight dynamic object).

3.4 Animation and rendering

Windmill. If the hierarchy load from `model_windmill.x` succeeds and reports animation sets, `Game::Update` advances `ID3DXAnimationController` time each frame (unless paused with `M`). If loading falls back to static geometry (OBJ or static `.x`), the renderer uses mesh subset draws; sail motion may be driven in code for that path—not via the keyframed controller.

Scene lighting. A day/night style effect moves the sun direction and interpolates clear and ambient colours (`src/Game.cpp`).

4 3D models, media, and authorship

4.1 Authorship statement

Vehicle, windmill, crate, and boulder meshes/materials under `Assets/models/` are original work by Guo Tsun Hei. Runtime terrain is generated procedurally. Sketchfab links in Stage 1 were references only, not shipped geometry.

4.2 Relation to Stage 2 submission

For markers: the **models and animation** for these assets (authored by Guo Tsun Hei) were **already finished** at the time of the **Stage 2** report submission. The Stage 2 development PDF (`docs/stageTwo.pdf`) did not include every *viewport screen capture* or *inline animation GIF* — that was a matter of what we chose to show in the written report, not the state of the work. The corresponding **asset and animation files** were nevertheless **included in the Stage 2 submission package**. This final report adds the stills, GIFs, and recordings in this section and in §4.5 as fuller documentation of the same content.

4.3 Integration notes

Jeep, crate, and boulder. `Game::Initialize` calls `LoadStaticModelCandidates` in `src/Game.cpp`; each candidate path is tried until one loads.

- **Jeep:** four attempts (bundled OBJ, flat replacement OBJ, root `model_jeep.obj`, then `model_jeep.x`).
- **Crate / boulder:** three OBJ attempts each (bundled, flat replacement, then root models folder). No `.x` list; if all fail, a textured box or sphere is built.

Windmill. The animated hierarchy at `Assets/models/model_windmill.x` is tried first. If that fails, static OBJs are tried in the same folder pattern as other props, with `model_windmill.x` last (no animation).

Scene setup. After load, meshes are placed in world space; `BuildReplacementMeshCorrection` adjusts orientation and scale where needed.

4.4 Static figure evidence

4.4.1 Authoring-tool viewports

These stills support texture work and scale; playable terrain remains procedural (see `src/Terrain.cpp`).

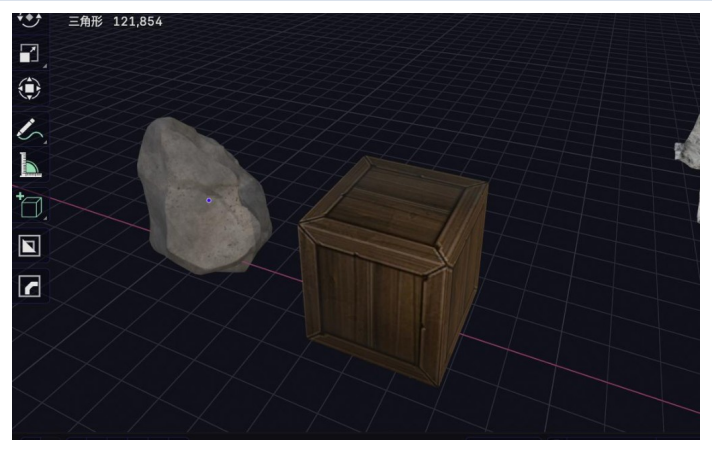


Figure 3: *Crate and boulder meshes in the authoring viewport.*

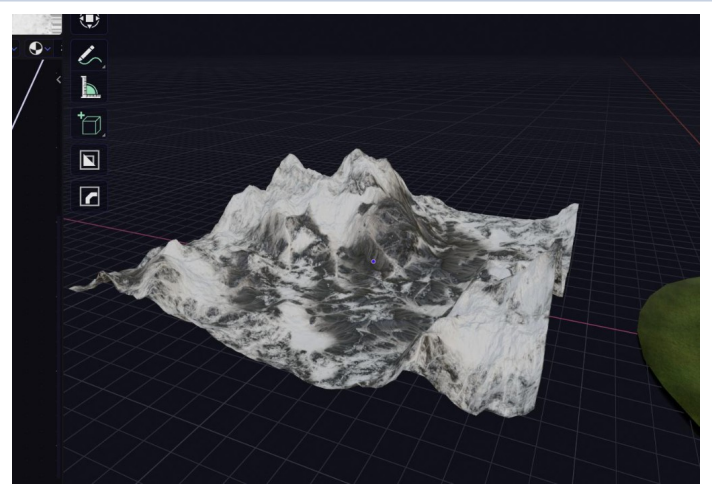


Figure 4: *Terrain sculpt and materials in the authoring tool (*playable hills are procedural in the exe*).*

4.4.2 Cinema 4D source frames (jeep and windmill)

Representative frames from screen recordings under `docs/media/`; GIF variants are listed in §4.5.

4.5 Video and README media

Recordings (paths in repo): `docs/media/c4d-car-viewport.mp4` and `docs/media/c4d-windmill-viewport.mp4`.
GIFs for the GitHub README live under `docs/images/`.

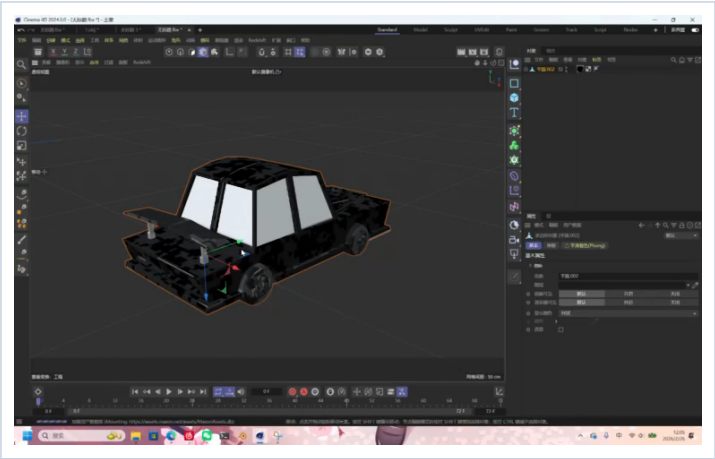


Figure 5: *Jeep source model: Cinema 4D viewport (frame from recorded session).*

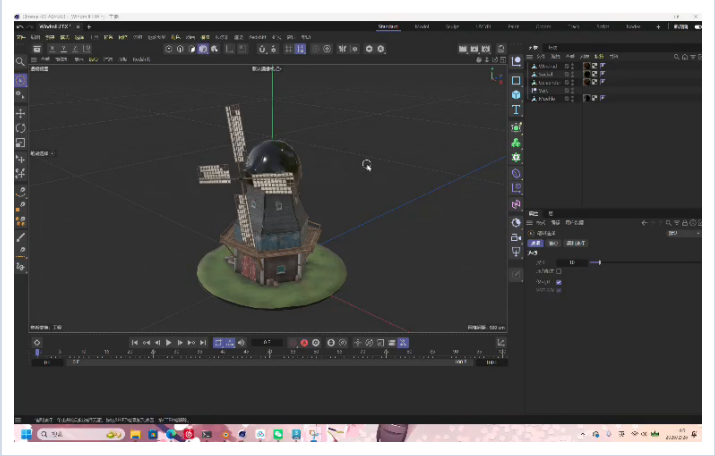
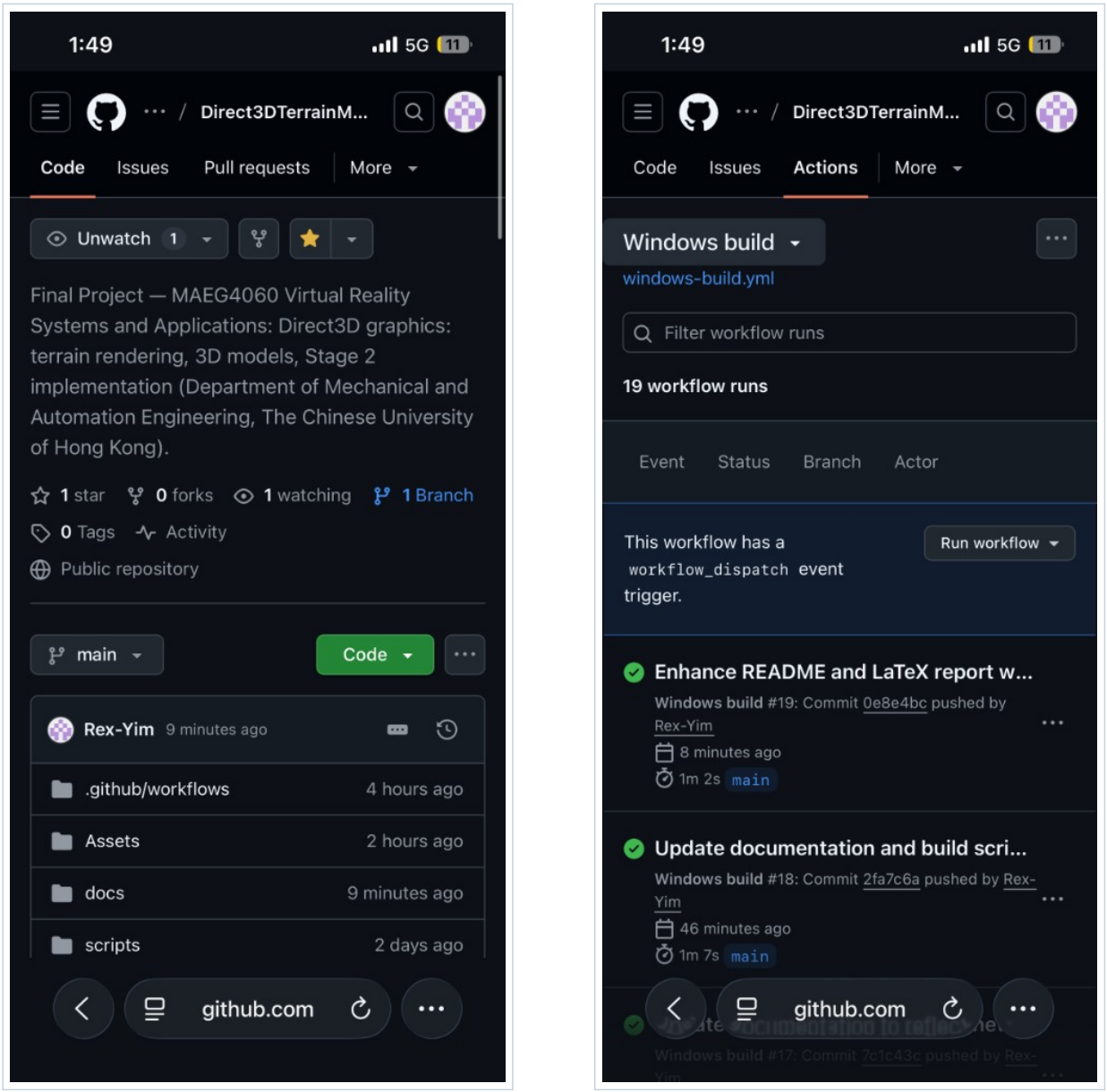


Figure 6: *Windmill source model: Cinema 4D viewport (frame from recorded session).*

4.5.1 Mobile browser views (public repository)

Phone screenshots below show how the **public GitHub repository** and its **README** look on a narrow viewport: landing page, Actions history, and scrolled README (runtime/CI embeds, then authoring tables). They complement Figures 1–6 with a typical mobile reading context.



(*a*) Repository home: description and file tree.

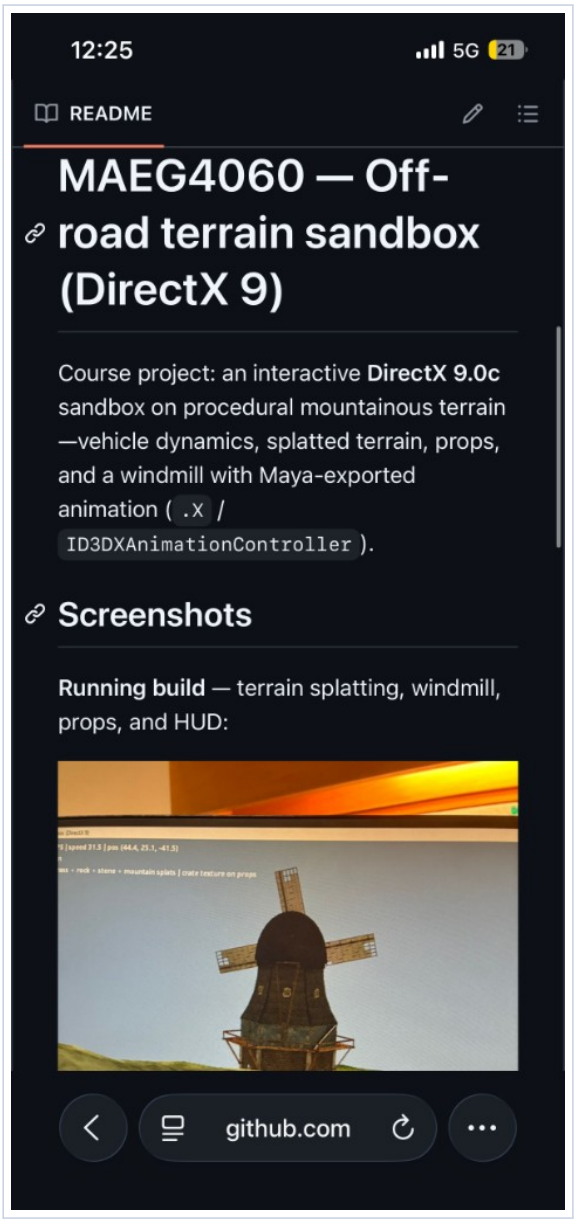
(*b*) `Actions`: `.github/workflows/windows-build.yml` run history.

Figure 7: *GitHub project page on a mobile browser: public landing view and Windows CI runs.*

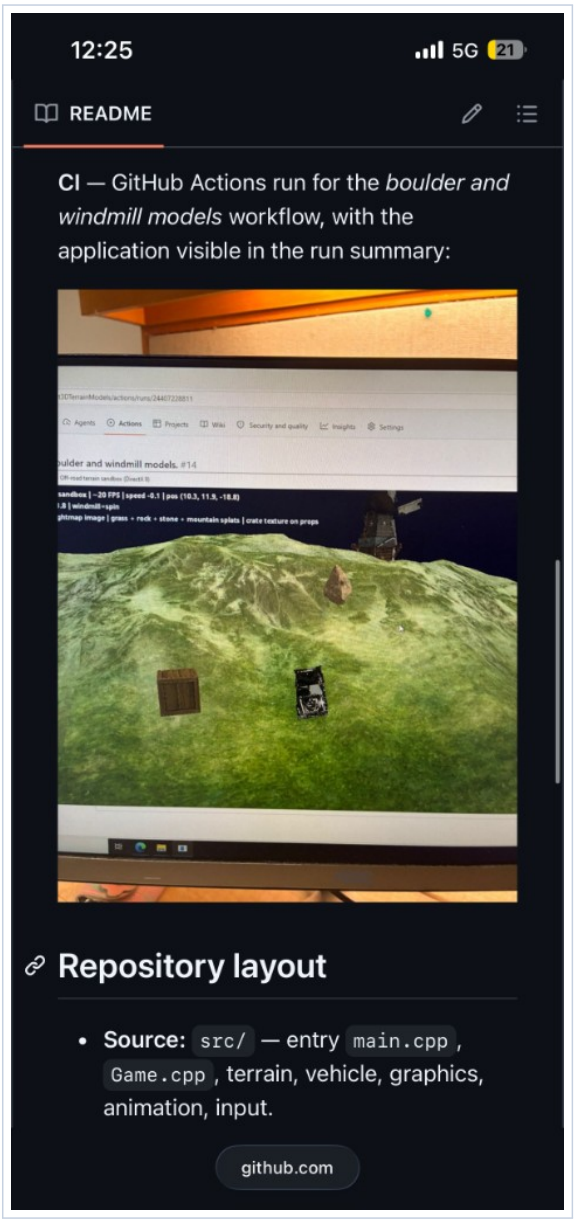
5 Build and packaging notes

5.1 Toolchain

Windows CMake, C++17, MSVC recommended. Target `maeg4060_stage2`. Links `d3d9`, `d3dx9`, `xinput9_1_0`.

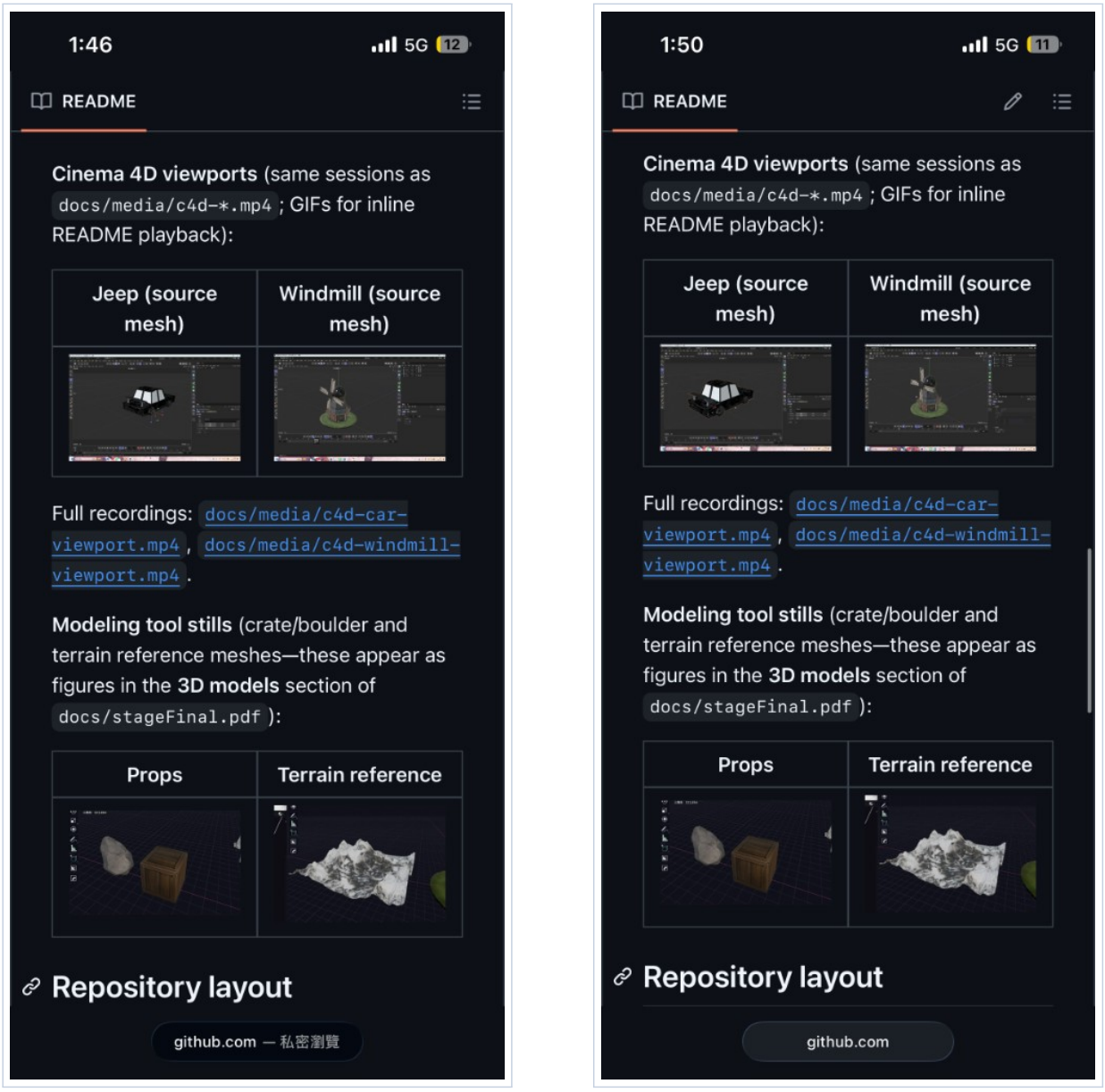


(*a*) README: **Screenshots** section (in-engine view).



(*b*) README: CI summary embed and start of **Repository layout**.

Figure 8: *README on a phone: embedded runtime imagery and continuous-integration evidence.*



(*a*) Cinema 4D viewport GIFs and modeling stills (tables).

(*b*) Same README region with recording links and layout header.

Figure 9: *README on a phone: authoring/source-mesh media referenced from §4.*

5.2 D3DX path

(1) Set `DXSDK_DIR` to the June 2010 DirectX SDK. **(2)** Or configure without it and let CMake obtain the NuGet D3DX package (see `CMakeLists.txt`).
Post-build or install steps may copy `D3DX9_43.dll` beside the executable when available.

5.3 Packaging target

Submit the full `Game_Build` folder (executable, `Assets/`, required DLLs), not the `.exe` alone.

6 Conclusion

The deliverable is a compact DirectX 9 sandbox: terrain navigation, input, collision, animation integration, and team-authored 3D content. Submission materials include this report, the standalone user guide PDF, staged course PDFs, source, `Assets/`, and build instructions; the public GitHub repository hosts the CI-backed tree used for integration screenshots.

Known limitations. There is no audio engine. The jeep has no per-wheel articulation in code. Terrain is procedural at runtime rather than a single imported artist heightmap (optional image heightfields are supported when assets exist). The boulder uses simplified sphere–box response rather than full rigid-body stacks. These bounds keep the codebase small while meeting sandbox and course demonstration goals.

Possible extensions. Import a large heightmap PNG, extend rigging for wheel bones, add richer rigid-body contact, and optional PBR or shader-based terrain beyond the fixed-function pipeline.

7 References

- MAEG4060 Virtual Reality Systems and Applications — course project description (2025–26, Semester 2), CUHK Faculty of Engineering.
- Microsoft Learn / MSDN: Direct3D 9, D3DX 9 utilities, XInput. `https://learn.microsoft.com/en-us/windows/win32/direct3d9`
- D3DX via NuGet package `Microsoft.DXSDK.D3DX`; see `CMakeLists.txt`.
- CMake documentation: `https://cmake.org/documentation/`
- GitHub repository and CI: `https://github.com/Rex-Yim/Direct3DTerrainModels`
- Local checkout: `src/`, `CMakeLists.txt`, `docs/User_Guide.pdf`.